

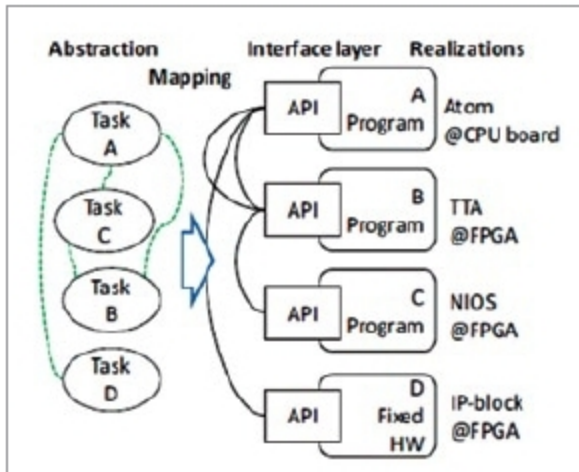


Fig. 1: Application tasks are implemented in processor programs and fixed-function hardware blocks using unified interface layer abstraction

- Address real-time aspects of system solution
- FPGA logic has flexible interconnect
- Trade data width with clock frequency using latency experimentation
- Allow experimentation with regards to changing microprocessor sub-system hardware designs; Altera FPGA under-the-hood
- Generic Linux interfaces are used; can be applied in any Linux system

Application portability is often provided by a compatibility layer API. However, traditional multi-computer APIs are too heavy and lack support for extreme heterogeneity like treating hardware IP blocks such as processors, where inter-core communication is expected to be faster and more reliable than inter-computer. Thus, MCAPI aims to be lightweight and efficient than other parallel computing APIs.

General code structure

```
include <omp.h>
main () {
  int var1, var2, var3;
  // Serial code
  // Beginning of parallel section.
  // Specify variable scoping
  #pragma omp parallel private(var1, var2)
  shared(var3) {
    // Parallel section executed by all threads
    // All threads join master thread and
    disband
  }
  Resume serial code
}
```

omp keyword distinguishes pragma as OpenMP pragma and is processed by OpenMP compilers.

Parallel region.

```
#include <omp.h>
main () {
  int nthreads, tid;
  /* Fork a team of threads
  #pragma omp parallel private(tid) {
  tid = omp_get_thread_num(); /* Obtain
  thread id */
  printf("Hello World from thread = %d\n",
  tid);
  if (tid == 0) { /* Only master thread does
  this */
  nthreads = omp_get_num_threads();
  printf("Number of threads = %d\n",
  nthreads);
  }
  } /* All threads join master thread and
  terminate */
}
```

Directive.

```
include <omp.h>
#define CHUNKSIZE 10
#define N 100
main () {
  int i, chunk;
  float a[N], b[N], c[N];
  for (i=0; i < N; i++)
    a[i] = b[i] = i * 1.0;
  chunk = CHUNKSIZE;
  #pragma omp parallel shared(a,b,c,chunk)
  private(i) {
    #pragma omp for schedule(dynamic,chunk)
    nowait
    for (i=0; i < N; i++)
      c[i] = a[i] + b[i];
  } /* end of parallel section */
```

Sections directive.

```
include <omp.h>
#define N 1000
main () {
  int i;
  float a[N], b[N], c[N], d[N];
  for (i=0; i < N; i++) {
    a[i] = i * 1.5; b[i] = i + 22.35;
  }
  #pragma omp parallel shared(a,b,c,d)
  private(i) {
    #pragma omp sections nowait {
      #pragma omp section
      for (i=0; i < N; i++)
        c[i] = a[i] + b[i];
      #pragma omp section
      for (i=0; i < N; i++)
        d[i] = a[i] * b[i];
    } /* end of sections */
  } /* end of parallel section */
}
```

Critical directive.

```
#include <omp.h>
main() {
```

```
int x;
x = 0;
#pragma omp parallel shared(x) {
  #pragma omp critical
  x = x + 1;
} /* end of parallel section */
}
```

Thread private.

```
include <omp.h>
int a, b, i, tid; float x;
#pragma omp threadprivate(a, x)
main () {
  /* Explicitly turn off dynamic threads */
  omp_set_dynamic(0);
  printf("1st Parallel Region:\n");
  #pragma omp parallel private(b,tid) {
    tid = omp_get_thread_num();
    a = tid; b = tid; x = 1.1 * tid + 1.0;
    printf("Thread %d: a,b,x= %d %d %f\n",tid,a,b,x);
  } /* end of parallel section */
  printf("Master thread doing serial work
  here\n");
  printf("2nd Parallel Region:\n");
  #pragma omp parallel private(tid) {
    tid = omp_get_thread_num();
    printf("Thread %d: a,b,x= %d %d %f\n",tid,a,b,x);
  } /* end of parallel section */
```

Reduction clause.

```
#include <omp.h>
main () {
  int i, n, chunk;
  float a[100], b[100], result;
  n = 100 ; chunk = 10 ; result = 0.0 ;
  for (i=0; i < n; i++) {
    a[i] = i * 1.0 ; b[i] = i * 2.0;
  }
  #pragma omp parallel for default(shared)
  private(i) schedule(static,chunk)
  reduction(+:result)
  for (i=0; i < n; i++)
    result = result + (a[i] * b[i]);
  printf("Final result= %f\n",result);
}
```

OpenMP

OpenMP is an API for shared-memory multi-processing programming in C, C++ and Fortran, on architectures ranging from desktops to super-computers (Unix and Windows NT platforms). It requires special support from the compiler, which, in turn, makes it easy to adopt.

Code can be compiled for serial execution, and parallelisation can be added gradually. MCAPI provides a limited number of calls with sufficient communication functionality